

生成式软件开发

2. 氛围编程的关键实践 (1)

—从Chat聊天到Agentic IDE

大纲

01 基于Chat聊天模式的Vibe Coding

探索通过自然语言对话与AI协作完成软件开发的基础方法

- 基于Chat聊天模式概述
- 会话（session）的概念
- 实践案例：天气小程序 & 排队预测
- 代码审查策略
- 提示词工程
- 基于Chat聊天模式的上下文管理

02 基于Agentic IDE的Vibe Coding

引入集成Agent功能的智能开发环境，实现自动化代码生成与项目管理

- Agent 的定义
- Workspace工作空间的概念
- 从"单点任务"到"项目"的转变
- 主流Agentic IDE工具生态

01

基于Chat聊天模式的 Vibe Coding

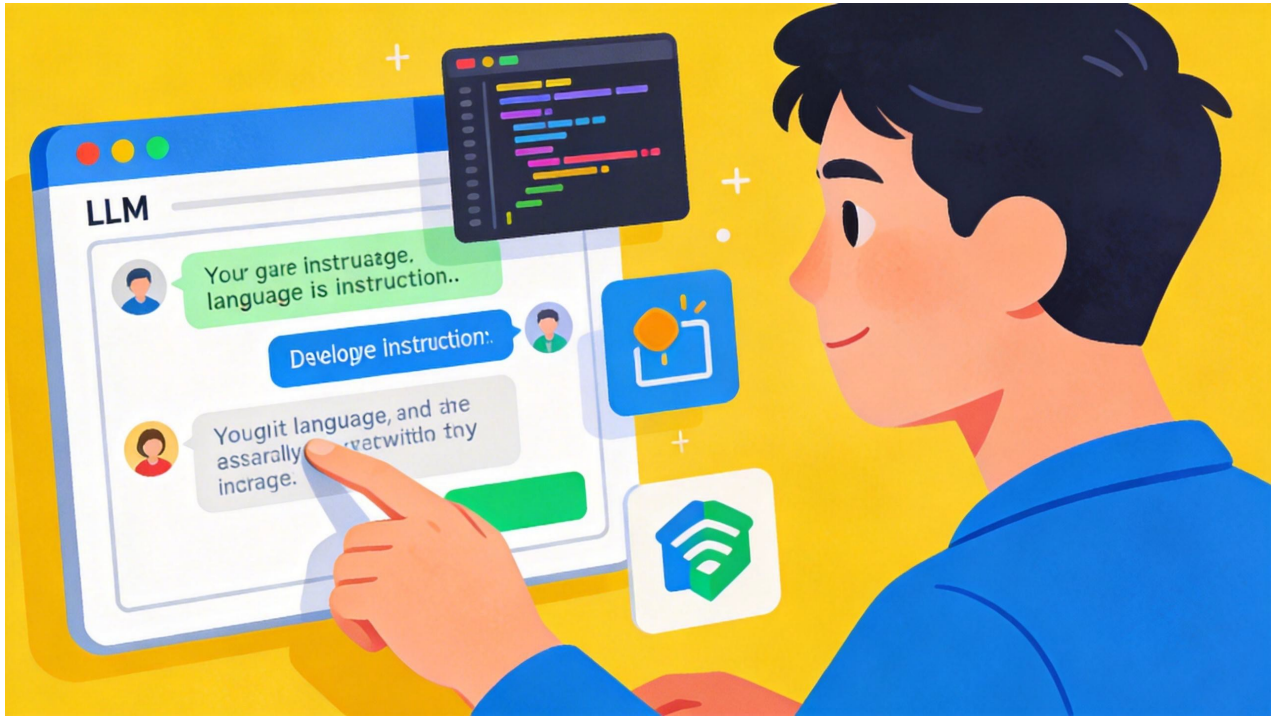
探索通过自然语言对话与AI协作完成软件开发的基础方法

Chat聊天模式概述

定义

Chat聊天模式是指在大模型提供的聊天窗口通过 **问答的方式** 进行软件开发，是生成式软件开发中 **最基础的交互方式** 。

通过在LLM客户端的对话界面进行**自然语言交流**来完成软件开发任务



核心思想

像与同事交流一样，使用**自然语言描述需求**，让AI理解并生成代码。无需编写特定语法或遵循严格的格式规范。

Chat模式核心特点



自由对话式

使用**自然语言描述需求**，无需编写特定语法或遵循严格的格式规范。就像与同事交流一样，直接表达你的想法和需求。

- **降低技术门槛，非技术人员也能参与**



基于会话的上下文理解

大模型能够**理解对话历史**，保持上下文连贯性。在同一会话(Session)内，AI记住之前的讨论内容，无需重复说明背景信息。

- **连续对话，逐步细化和完善需求**



即时反馈

问题与回答**交替进行**，快速迭代。提出问题后立即获得AI的响应，可以迅速验证想法和调整方向。

- **快速原型，即时看到结果**



多轮交互

通过**多轮对话**逐步完善代码。第一轮生成基础框架，第二轮添加功能，第三轮优化细节，循序渐进地构建完整应用。

- **渐进式开发，降低复杂度**

会话（Session）的概念

什么是会话?

一次对话的完整上下文环境，包含用户的所有提问和AI的所有回复。会话是Chat模式中的 记忆单元，让AI能够理解和关联对话内容。

会话 = 记忆 + 上下文 + 连续性

会话管理建议

⚠ **重要代码及时保存**
会话可能因超时或刷新而重置

⚠ **新会话需重新描述**
之前的上下文不会自动带入

⚠ **定期整理关键信息**
避免会话过长导致性能下降

上下文记忆示例

U User

帮我写一个Python函数，计算两个数的和

AI Assistant

```
def add(a, b):
return a + b
```

U User

加上类型检查

AI Assistant

```
def add(a, b):
if not isinstance(a, (int, float))...
return a + b
```

在同一会话内，AI能够记住之前对话的内容

示例1：天气新闻应用

需求描述

构建一个HTML页面，每次打开能够获取 **当前所在位置的天气** 和 **最新事件**。

实现功能

地理位置定位
自动获取用户当前位置

实时天气数据
显示温度、湿度、风速

本地新闻资讯
展示当前位置相关新闻

学习价值

展示如何通过Chat模式快速构建一个**网页应用**，涵盖地理位置、天气、资讯等API调用、数据展示等核心功能。



示例2：排队等待时间预测

需求描述

排队等待时间预测程序。初始化时输入号码和当前叫号，每听到一个号就点击按钮计数，程序根据数据给出等待时间预测。

智能预测算法

1. **移动平均法**
   计算最近N次叫号的平均时间间隔
2. **动态调整**
   根据实时数据更新预测模型
3. **置信度评估**
   基于数据量给出预测可信度

学习价值

展示在Chat模式中，如何通过 **多轮交互** 构建一个Web应用。



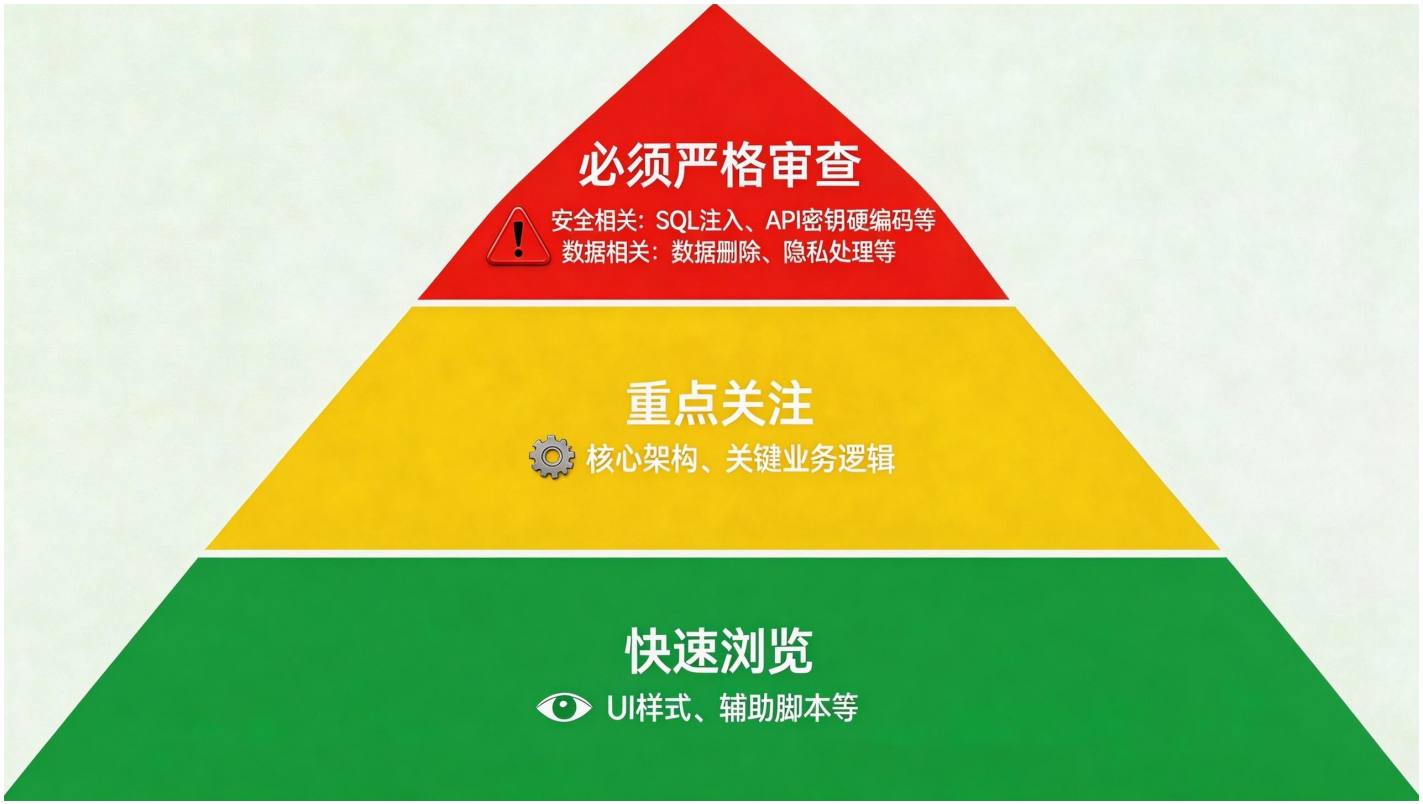
需要审查生成的代码吗？

为什么需要审查？

- AI可能生成不安全或低效的代码
- 业务逻辑理解可能存在偏差
- 安全漏洞和数据风险需要人工把关
- 代码质量和可维护性需要验证

审查原则

在氛围编程中，虽然我们鼓励"让AI写代码"，但这并不意味着**完全放弃审查**。根据风险等级，建议采用**分级审查策略**。



三级审查策略，合理分配审查精力

必须严格审查

安全相关

API密钥硬编码

确保敏感密钥存储在环境变量或安全密钥管理服务中

用户认证逻辑

检查登录、注册、密码重置等认证流程的安全性

文件权限设置

验证文件上传、下载、删除的权限控制是否严格

SQL注入风险

检查所有数据库查询是否使用参数化查询，避免字符串拼接SQL

数据相关

数据删除/修改

确保删除和修改操作有确认机制和日志记录

隐私数据处理

检查PII数据是否加密存储和传输

备份恢复逻辑

验证数据备份策略和恢复流程的可靠性

原则：一旦出错，后果不可逆或代价极高

如：天气新闻应用示例

实现功能

地理位置定位

自动获取用户当前位置

实时天气数据

显示温度、湿度、风速

本地新闻资讯

展示当前位置相关新闻

安全设计：

NewsAPI密钥通过输入框填写，避免硬编码在代码中。



重点关注&快速浏览

重点关注

核心架构

模块划分、接口定义、数据结构设计

关键业务逻辑

复杂的计算公式、状态机流转、支付金额计算

原则：决定了系统的稳定性和可扩展性

快速浏览

UI/样式代码

CSS调整、布局微调（直接看预览效果）

辅助脚本

一次性数据清洗脚本、简单的工具函数

样板代码

Getter/Setter、简单的CRUD接口

原则：仅验证运行结果，而非逐行读代码

提示词工程

关键洞察

"怎样说"——把需求准确、高效地传达给大模型是这个过程中的关键环节。

提示词工程正是把 "模糊想法" 翻译成 "可执行指令" 的关键桥梁。

→ 优秀的提示词 =
清晰的需求 + 完整的上下文 + 明确的约束

提示词质量影响

提示词工程是用户与AI协作的"**指令接口**"，直接决定了代码生成的准确率。

定义

提示词工程是通过 **精心设计、优化和精炼输入文本 (Prompt)**，引导大模型（LLM）生成更准确、更高质量输出的技术与方法。

核心目标

1. **降低歧义**
   消除模糊表达，明确需求边界
2. **提供上下文**
   给出必要的背景信息
3. **提供约束**
   使模型输出符合预期格式和标准

提示词工程核心策略

角色设定(Role Prompting)

明确告诉AI你希望它**扮演的角色**，例如"你是一位Python开发者"、"你是一位前端架构师"。

- 通过角色约束让模型优先关注相关信息，减少无关输出。

结构化提示词(Structured Prompting)

通过**结构化的格式和内容**避免模糊，让需求传达更加清晰、准确、易于执行。

- **结构化格式**：使用XML/Markdown等格式明确区分不同内容块**结**
- **构化内容**：将需求分成多个小点，清晰列出功能、约束等

示例提示(N-Shot Prompting)

通过提供**一个或多个"输入-输出"示例**，让模型快速模仿格式、风格和逻辑，比单纯的规则描述更直观有效。

- "Show, don't just tell."（演示，而不只是讲述）

思维链(Chain-of-Thought, CoT)

要求模型在给出最终答案前先进行**分步骤思考**，再输出结论。

- 关键技巧："Let's think step by step"（请一步步思考）

策略一：角色设定 (Role Prompting)

什么是角色设定?

明确告诉AI你希望它 **扮演的角色** ，例如"你是一位Python开发者"、"你是一位前端架构师"。

为什么需要角色设定?

收敛上下文 (Focus)

通过角色约束让模型优先关注相关信息，减少无关输出。

确立标准 (Standard)

暗示输出代码应符合该角色的专业标准（代码风格、错误处理）。

消除歧义 (Disambiguation)

确保术语被理解为编程领域的概念，而非日常用语。(比如Cookie)

示例

无角色设定

帮我写个排序算法

有角色设定

你是一位资深算法工程师，请用Python实现一个高效的快速排序算法，要求时间复杂度为O(n log n)，并添加详细注释。

研究支持

研究表明，赋予特定**专家角色**能显著提升模型解决特定领域问题的准确率。

常用角色示例



策略二：结构化提示词

什么是结构化提示词?

通过 **结构化的格式和内容** 避免模糊，让需求传达更加清晰、准确、易于执行。

结构化格式(Structured Format)

使用 XML、JSON、Markdown 等格式明确区分不同内容块

?为什么有效?

- ✓ **边界清晰**：明确区分“代码”、“需求”、“数据”等不同类型的信息。
- ✓ **解析准确**：大模型对结构化数据的注意力机制更强。
- ✓ **减少幻觉**：防止模型混淆指令与内容

结构化内容 (Structured Content)

将需求分成 多个小点，通过特定的结构（如标题、小节）将自然语言指令进行逻辑分层，清晰列出功能、约束等

?为什么有效?

- ✓ **降低认知负荷**：模型不需要在一段长文本中费力寻找关键信息。
- ✓ **强制清晰思考**：迫使提问者将模糊的想法拆解为具体的“背景”、“目标”、“约束”。
- ✓ **便于迭代**：当需求变更时，只需修改对应的小节，而不需要重写整段话。

结构化提示词让AI更容易**理解你的意图**，生成更符合预期的代码。

结构化格式示例

使用XML标签格式

```
<tech_stack>
language: Python 3.12
framework: FastAPI
deps: pydantic v2
</tech_stack>

<requirements>
- 实现二分查找
- 返回索引；不存在返回 -1
</requirements>
```

非结构内容与结构化内容对比

原始Prompt (模糊、非结构化)

我现在在医院排队做CT，人很多，估计要等数小时。现在希望能做一个预测何时能排到队的HTML小程序，初始化时输入我的号码和当前叫号，接下来每听到一个号就点击一个按钮计数。程序在有了一定的数据后给出推断我还需要等多长时间。

- 信息混杂，结构不清
- 需求边界模糊
- 缺少技术约束
- 难以准确执行

结构化Prompt （清晰、易执行、更准确）

角色

你是一位资深的前端开发工程师，擅长开发移动端友好的单页应用。

背景

我正在医院排队等待检查，需要一个工具来根据当前的叫号速度，预估我还需要等待多久。

目标

请编写一个单文件的HTML小程序（包含CSS/JS），用于追踪排队进度并预测时间。

功能需求

1. **初始化设置**：允许用户输入「我的号码」和「当前叫号」
2. **核心交互**：提供一个显眼的「过号」按钮
3. **预测逻辑**：采用移动平均法计算等待时间
4. **状态显示**：实时显示当前叫号、预计等待时长

……

- 逻辑清晰，层次分明
- 需求边界明确
- 包含技术约束
- 易于准确执行

结构化内容完整示例

完整Prompt示例

角色
你是一位资深的前端开发工程师，擅长开发移动端友好的单页应用。

背景
我正在医院排队等待检查，需要一个工具来根据当前的叫号速度，预估我还需要等待多久。这个工具将在手机浏览器上运行。

目标
请编写一个单文件的HTML小程序（包含CSS/JS），用于追踪排队进度并预测时间。

功能需求

1. **初始化设置**：允许用户输入「我的号码」和「当前叫号」
2. **核心交互**：提供一个显眼的「过号」按钮，每点击一次，「当前叫号」加1，并记录当前时间戳
3. **预测逻辑**：
   - 采用**移动平均法**：优先计算最近10次叫号的平均时间间隔
   - 预测「剩余等待时间」= (我的号码 - 当前叫号) * 移动平均时间
   - 至少有3次点击数据后，才开始显示预测结果
4. **状态显示**：实时显示当前叫号、我的号码、预计还需要等待的时长（分钟）、预计叫号时间点

技术要求

- 单文件HTML（内嵌CSS和JS）
- 移动端友好，按钮要大
- 使用localStorage保存数据，刷新页面不丢失

约束

- 代码简洁，易于理解
- 添加必要的注释

结构化要素

1. **角色** - 明确专家身份 **背**
2. **景** - 说明使用场景 **目标**
3. - 明确最终产出 **功能需**
4. **求** - 详细功能点**技术要**
5. **求** - 技术栈约束**约束** - 额
6. 外限制条件

关键优势

- 降低认知负荷
- 强制清晰思考
- 便于需求迭代

将模糊想法转化为可执行指令

策略三：示例提示（N-Shot Prompting）

什么是N-Shot提示?

通过提供 一个或多个"输入-输出"示例 ，让模型快速模仿格式、风格和逻辑，比单纯的规则描述更直观有效。

核心思想

"Show, don't just tell."

(演示，而不只是讲述)

适用场景

- 复杂格式要求
- 特定代码风格
- 冷门领域知识
- 需要举一反三

三种示例模式

0 Zero-shot (零样本提示)

不给示例，直接问

适合通用简单任务

1 One-shot (单样本提示)

给1个示例，确立格式

适合规范格式

N Few-shot (少样本提示)

给2-3个示例，覆盖边缘情况

适合复杂逻辑

零样本提示 (Zero-shot)与单样本提示 (One-shot)

0 Zero-shot (零样本)

特点

直接提问，不给任何示例

示例

"写一个提取邮箱的正则表达式。"

适用场景

- ✓ 简单、通用的任务
- ✓ 不需要特定格式
- ✓ 快速获取基础代码

适合快速原型和通用查询

1 One-shot (单样本)

特点

提供1个示例，确立格式和风格

示例

请参考以下代码风格，写一个提取邮箱的正则表达式：

示例：提取手机号 -> r'^1[3-9]\d{9}$'

适用场景

- ✓ 需要规范格式
- ✓ 特定代码风格
- ✓ 确立基准标准

通过示例传递隐式约束

少样本提示 (Few-shot)

什么是Few-shot?

提供 2-3个示例 ，帮助模型理解复杂逻辑或边缘情况。通过多个示例覆盖不同场景，让模型能够举一反三。

示例：Emoji 风格转换

Prompt: 把“哈利波特”转换成 Emoji。
输出: 🧙 (通常很简单)

Prompt:
将哈利波特这部电影标题转换为 Emoji 组合，要体现剧情精髓。

示例 1：泰坦尼克号 -> 🚢💥🌊💑🎻🌊
示例 2：黑客帝国 -> 💊🕶️🟢💻🐇🔫

输出: ⚡👓🧙🦉🚂🏰🧹

适用场景

逻辑复杂
需要理解多层规则的任务

边缘情况
需要覆盖多种特殊场景

举一反三
需要模型推理新情况

设计原则

- 示例要覆盖主要场景
- 包含边缘情况和异常
- 保持示例格式一致性
- 示例数量2-3个为宜

通过示例让模型"学会"规则

策略四：思维链 (Chain-of-Thought)

什么是思维链?

要求模型在给出最终答案前先进行 分步骤思考 ，再输出结论。

"Let's think step by step"

(请一步步思考)

作用

提高准确率

显著提高处理复杂逻辑、算法设计的准确率

为什么有效

强迫模型先“**想清楚**”再“**写代码**”， 避免直接生成似是而非的错误代码。

对比示例：重构遗留代码

普通Prompt

"这段Python代码运行太慢了，请帮我优化一下。"

CoT Prompt

"这段Python代码运行太慢了。

请一步步思考：

1. 先分析当前代码的时间复杂度？瓶颈在哪里？
2. 列出2-3种优化方案，对比优缺点
3. 选择最优方案，给出重构后的代码"

提示词其余策略（会在后续内容中涉及）

提供参考文本 (Provide Reference Text)

大模型有时会编造答案（幻觉），提供由相关性检索（RAG）获取的参考文本，可以显著提高准确性。

拆分复杂任务 (Split Complex Tasks)

复杂任务不仅错误率高，而且难以通过单一 Prompt 覆盖所有边缘情况。拆分任务可以降低每一个步骤的认知负担。

使用外部工具 (Use External Tools)

弥补大模型的短板（如数学计算不准、无法获取实时信息、无法运行代码）。比如让模型写 Python 代码并执行它来做数学计算，而不是让模型自己算。

系统地测试变更 (Test Changes Systematically)

Prompt 的修改可能会导致“按下了葫芦浮起了瓢”。需要建立评估集（Evals），确保新的Prompt在整体表现上是提升的。

理解Token

什么是Token?

Token是AI处理文本的 最小单位，也是计费的"货币"。AI不直接阅读文字，而是将其拆解为Token进行处理。

Token的特点

并非字符或单词

使用BPE（Byte Pair Encoding）等分词算法进行拆解

中英文差异

英文单词是一个Token

中文一个汉字可能对应平均1.2个Token(含生僻字)

计费单位

Token数量直接影响API调用成本

节省Token = 节省金钱 + 提升速度



Token计算示例

"Hello World!" "	~3 Tokens
你好世界"	~4 Tokens
"function add(a, b)"	~7 Tokens

AI如何思考与表达？

核心原理

AI根据已给出的内容，预测下一个最可能出现的字词（自回归生成）。这是一个概率预测过程，而非真正的"理解"。

生成步骤演示

1 理解输入

"帮我写一个Python二分查找函数"

2 预测概率

计算下一个词的概率：

def	**80%**
function	15%
class	5%

→ **选择 def**

3 更新上下文

Context: "帮我写一个Python二分查找函数 **def**"

4 循环生成

继续预测：

binary_search(a, b):

...

直到完成完整函数

AI的"思考"本质是**概率预测**，而非真正的理解。提供清晰的上下文能显著提高预测准确性。

理解上下文窗口

定义

模型一次能处理的**最大Token数量**（包括输入和输出）。这是AI的"工作记忆"上限。

"短期记忆"的极限

就像人的工作记忆，超过限制的内容会被**"挤出"或截断**。

这解释了为什么长对话中AI会"忘记"之前的设定

Chat模式中的影响

Chat 聊天模式中，上下文窗口是由会话决定的。随着对话轮次增加，会话内容越来越多，形成长会话问题。

长会话的问题

理解延迟

模型需要处理更多上下文，响应变慢

生成质量下降

过长会话导致代码质量下降，因需综合考虑更多的上下文信息。

资源消耗增加

需要更多计算资源，需要更高成本

节省Token = 节省金钱 + 提升速度 + 避免遗忘

Chat模式的上下文管理策略

	管理方式	是否全发历史	优缺点
1	**完整Message List** (system + 所有历史user/assistant + 新消息)	是 (但限context window内)	简单直接 但增加成本，长对话容易超限
2	**Sliding Window + Truncation** 只保留最近N 条消息（或最近X tokens）	否 只发送最近部分	成本低 可能丢失早期重要信息
3	**自动Summarization / Compaction** 每隔几轮把旧历史总结成"摘要"，替换原始消息	否 发送压缩后的内容	保留连贯性，Token少 需要额外计算 (通常平台后台自动完成)

对于长对话，不同的管理方式对于会话的成本和精确度会有比较大的影响

Vibe Coding与上下文管理

Vibe Coding的核心

Vibe Coding的过程就是通过提示词告诉大模型**需求描述**和实现需求需要的相关**知识**，让大模型根据这些信息生成符合需求的代码。

关键能力：**知道对于相关问题而言，什么是需要额外附加的相关知识，并有效地传递给大模型**

上下文类型

需求描述
用户想要什么
"做一个待办事项应用"

领域知识
相关的业务背景
"餐厅点餐系统"

技术约束
技术栈、依赖、环境
”用Python + Flask“，”要有直观的可视化“

代码上下文
现有代码的结构
"auth.py已有用户认证模块"

业务规则
业务逻辑和流程
"VIP用户享受8折优惠"

数据约束
数据格式、边界
"用户年龄必须>18"

高效Vibe Coding本质上是高效的上下文管理

上下文越完整、越准确 → 生成的代码越符合预期

并不是越多越好，而是在完整、准确的前提下**越少越好**

Chat模式的优缺点

优点

1. **低门槛**
   无需配置开发环境，会聊天即可开始编程
2. **快速原型**
   几句话就能生成可运行的代码，适合快速验证想法
3. **学习友好**
   适合编程初学者理解代码逻辑和编程概念

局限性

1. **不能直接创建和修改代码文件**
   代码产物需要手动落到文件中，后续修改也需要手动同步
2. **代码分散**
   生成多个文件时需要手动整理和组织
3. **调试过程繁琐**
   运行出错需要截图或手动复制错误信息反馈
4. **跨会话上下文无法保持**
   新会话不会自动带入之前Session的上下文，需要手动复制粘贴

Chat模式适合**快速原型**和**学习探索**场景

对于**复杂项目**，Chat模式的局限性会显著影响开发效率

示例：Chat 聊天模式的局限性

Python数据分析脚本

角色

你是一位 Python 数据分析师。请帮我写一个数据分析脚本。

功能需求

1.读取 CSV 文件（文件路径通过命令行参数传入）

2. 自动识别数值列

3. 计算并输出每个数值列的统计信息：均值、中位数、标准差、最小值、最大值、样本数

4. 生成柱状图展示各数值列的均值对比，保存为 chart.png

技术要求

- 使用 pandas 读取 CSV
- 使用 matplotlib 生成图表
- 添加命令行参数解析（--file 指定CSV文件路径）
- 添加基本的错误处理

约束

- 先给出完整代码，然后解释关键部分。



Chat 示例



本地文件

02

基于Agentic IDE的 Vibe Coding

探索集成Agent功能的智能开发环境，实现自动化代码生成与项目管理

Agentic IDE：智能开发环境

什么是Agent

是一个通过**传感器**(Sensors)感知环境，通过**执行器**(Actuators)作用于环境的计算实体。

在现代AI Agent的框架中，Agent是一个能够理解**目标**、**制定步骤**并**调用工具**去执行任务的AI执行体。

不只是“回答问题”，而是会主动完成一整段工作流：例如读取代码、修改文件、运行命令、检查结果，并根据反馈继续迭代直到任务完成。

Agentic IDE

是一种集成了Agent功能的开发环境，通过与Agent交互实现自动化的**代码生成**、**调试**和**部署**。

Agentic IDE 通常提供一个可视化的界面，用于配置 Agent 的行为和参数，同时也展示 Agent 执行的结果和日志。



Workspace（工作空间）

Workspace（工作空间）

用于组织和管理Agent执行的任务和相关文件。

- Agent可以在Workspace中**执行任务**，如读取代码、修改文件、运行命令等。
- Workspace 还可以用于**存储 Agent 执行的结果**，例如生成的代码、运行日志等。

Workspace是Agentic IDE 中的一个概念，也称Workspace为**项目（Project）**

从"单点任务"到"项目"

单点任务 (Chat)

- "帮我翻译这句话。"
- "给我讲个笑话。"

特点：随用随走，不需要记忆上下文，不需要处理多个文件

项目 (Project)

- "帮我策划一次年会" → 人员名单、预算表、场地照片、PPT
- "帮我写一个个人网站" → HTML、CSS、图片、JavaScript

特点：文件之间有关系，需要长期保存，需要反复修改

实践示例1：整理目录中的文件

需求描述

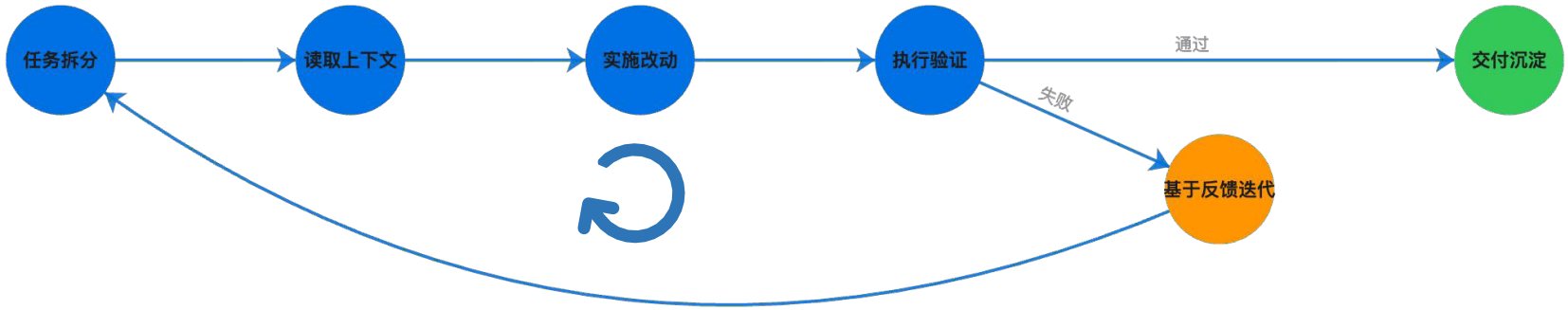
将“论文评审”目录中的“初评”、“终评”目录下的文件根据学生姓名重组，分别copy到相应以学生姓名命名的目录中。

引言案例

即便一个简单的任务，使用 **Agentic IDE** 也可以高效地完成。

Agentic IDE工作闭环

任务拆分 → 读取上下文 → 实施改动 → 执行验证 → 基于反馈迭代 → 交付沉淀



1 任务拆分

把目标拆成可执行的小步骤，明确每一步的验收标准

2 读取上下文

定位相关文件、接口和配置，避免在错误位置改动

3 实施改动

按步骤修改代码或文档，保证改动范围可控

4 执行验证

运行命令、测试与类型检查，用结果验证改动是否正确

5 基于反馈迭代

根据报错和测试结果继续修复，直到结果收敛

6 交付沉淀

总结改动、风险与后续事项，形成可追踪的交付记录

闭环标准：需求满足 + 关键验证通过 + 可复现执行流程

Agentic IDE—复杂项目的超级助手

当任务升级为复杂项目时，Agentic IDE提供更多价值

全局感知

Context Awareness

Chat窗口

每次提问都需要把相关文件代码贴给它，否则它不知道你改在哪里？

Agentic IDE

整个Workspace成为上下文，知道style.css定义了什么颜色，index.html引用了哪些脚本。

多文件联动

Multi-file Orchestration

Chat窗口

很难处理跨文件修改，如“增加用户登录功能”，需要涉及到修改 HTML结构、CSS 样式、JS 逻辑，甚至后端接口。

Agentic IDE

一次性规划并修改多个文件，一气呵成

渐进式交付

Progressive Delivery

Chat窗口

一次性生成一大段代码，很难维护

Agentic IDE

支持基于Workspace的迭代，每一步的成果都保存在 Workspace 中，稳扎稳打

Agentic IDE将单点对话升级为项目级协作，实现真正的智能开发

实践示例2：数据处理

工作闭环

实现一个简单的 **Python** 数据处理脚本

编写案例

工作闭环， **Agentic IDE**会自己测试代码是否运行正确，自动修复错误。

Agentic IDE 中大模型选择与工具生态

Agentic IDE与大模型的关系

Agentic IDE是智能体（Agent）
接受指令、管理Workspace、与大模型交互、执行任务，与用户交互

大模型是其"大脑"
提供强大的文本生成能力，根据指令生成代码和文本

集成多个大模型
用户可根据项目需求选择不同模型（GPT、Claude、Kimi等），以获得最佳生成效果。

大模型选择策略

先根据任务目标选择模型，再根据反馈动态切换

复杂推理优先
架构设计、疑难Bug、跨模块重构，优先选择推理强的模型

速度与成本优先
样板代码、文案整理、轻量改动，优先选择响应快，成本低模型

上下文长度优先
仓库大、文件多、上下文长，优先选择长上下文模型

推荐：高能力模型定方案 → 均衡模型实现 → 轻量模型修改

主流Agentic IDE工具对比

工具	厂商/开源	形态	代表性特点
TRAE	商业 (ByteDance)	独立IDE	项目级上下文 + 多步任务执行 (编辑文件/运行命令/验证)
Cursor	商业 (Anysphere)	独立IDE	Repo-aware，多文件改动 Agent/Compose工作流
Qoder	商业	独立IDE/插件	面向真实项目的 Agentic工作流 强调大仓库上下文理解
OpenCode	开源	CLI/TUI	终端里的编码Agent：读仓库、改文件、跑指令、会话管理
Claude Code	商业 (Anthropic)	CLI/IDE集成	在终端执行Agent：读仓库、改文件、Git工作流

除了TRAE，业界还有Cursor、Qoder、OpenCode、Claude Code等多种选择

生成式软件开发

End

2.氛围编程的关键实践 (1)
—从Chat聊天到Agentic IDE